

Expt 8

Multirate Signal Processing

8.2 Polyphase Implementation of Decimation and Interpolation Filters Using MATLAB

Objectives:

1. To perform sampling rate reduction of a discrete-time signal using a polyphase FIR decimation filter with a decimation factor of 2.
2. To perform sampling rate increase of a discrete-time signal using a polyphase FIR interpolation filter with an interpolation factor of 2.
3. To reconstruct the original signal from the decimated signal through interpolation and compare the reconstructed signal with the original input signal.
4. To demonstrate the computational efficiency and effectiveness of polyphase filter structures in multirate digital signal processing applications.

Procedure:

1. Generate the Input Signal
2. Design the FIR Low-Pass Filter
3. Perform Polyphase Decimation Choose the decimation factor $M = 2$.
4. Perform Polyphase Interpolation Choose the interpolation factor $L = 2$.
5. Analyze the Signals Plot the original signal, decimated signal, and interpolated signal in the time domain.
6. Analyze the Frequency Response Obtain the frequency response of the FIR filter using the `freqz()` function.
7. Verify Signal Lengths Display the lengths of the original, decimated, and interpolated signals.
8. Verify successful decimation by a factor of 2. Verify successful interpolation by a factor of 2.
9. Conclude that polyphase implementation achieves efficient multirate processing with reduced computational complexity.

Program (Matlab coding):

```
clc;  
  
clear;  
  
close all;
```

```
%% Input Signal
```

```
Fs = 1000;          % Sampling Frequency
```

```
t = 0:1/Fs:1;       % Time Vector
```

```
% Test Signal
```

```
x = sin(2*pi*50*t) + 0.5*sin(2*pi*120*t);
```

```
%% FIR Low-Pass Filter Design
```

```
N = 20;             % Filter Order
```

```
Fc = 0.4;           % Normalized Cutoff Frequency
```

```
h = fir1(N, Fc, hamming(N+1));
```

```
%% Decimation Factor
```

```
M = 2;
```

```
% Polyphase Decimation
```

```
y_dec = upfirdn(x, h, 1, M);
```

```
%% Interpolation Factor
```

```
L = 2;
```

```
% Polyphase Interpolation
```

```
y_int = upfirdn(y_dec, h, L, 1);
```

```
%% Adjust Length for Comparison
```

```
if length(y_int) > length(x)
```

```
    y_int = y_int(1:length(x));
```

```
elseif length(y_int) < length(x)
```

```
    x = x(1:length(y_int));
```

end

%% Time Axes

t1 = (0:length(x)-1)/Fs;

t2 = (0:length(y_dec)-1)/(Fs/M);

t3 = (0:length(y_int)-1)/Fs;

%% BEST GRAPH OUTPUT

figure('Name', 'Polyphase Decimation and Interpolation', ...
 'NumberTitle', 'off');

% Original Signal

subplot(4,1,1);

plot(t1, x, 'b', 'LineWidth', 1.5);

title('Original Input Signal');

xlabel('Time (s)');

ylabel('Amplitude');

grid on;

% Decimated Signal

subplot(4,1,2);

stem(t2, y_dec, 'r', 'filled');

title('Decimated Signal (M = 2)');

xlabel('Time (s)');

ylabel('Amplitude');

grid on;

% Interpolated Signal

subplot(4,1,3);

plot(t3, y_int, 'k', 'LineWidth', 1.5);

title('Interpolated/Reconstructed Signal (L = 2)');

```

xlabel('Time (s)');
ylabel('Amplitude');
grid on;

% Comparison
subplot(4,1,4);
plot(t1, x, 'b', 'LineWidth', 1.5);
hold on;
plot(t3, y_int, 'r--', 'LineWidth', 1.5);
title('Comparison of Original and Reconstructed Signals');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Original Signal', 'Reconstructed Signal');
grid on;

%% Display Information
disp('-----');
disp('POLYPHASE DECIMATION & INTERPOLATION');
disp('-----');

fprintf('Original Signal Length = %d\n', length(x));
fprintf('Decimated Signal Length = %d\n', length(y_dec));
fprintf('Interpolated Signal Length = %d\n', length(y_int));

%% Separate Figure for Better Visualization
figure;

plot(t1, x, 'b', 'LineWidth', 2);
hold on;
plot(t3, y_int, 'r--', 'LineWidth', 2);

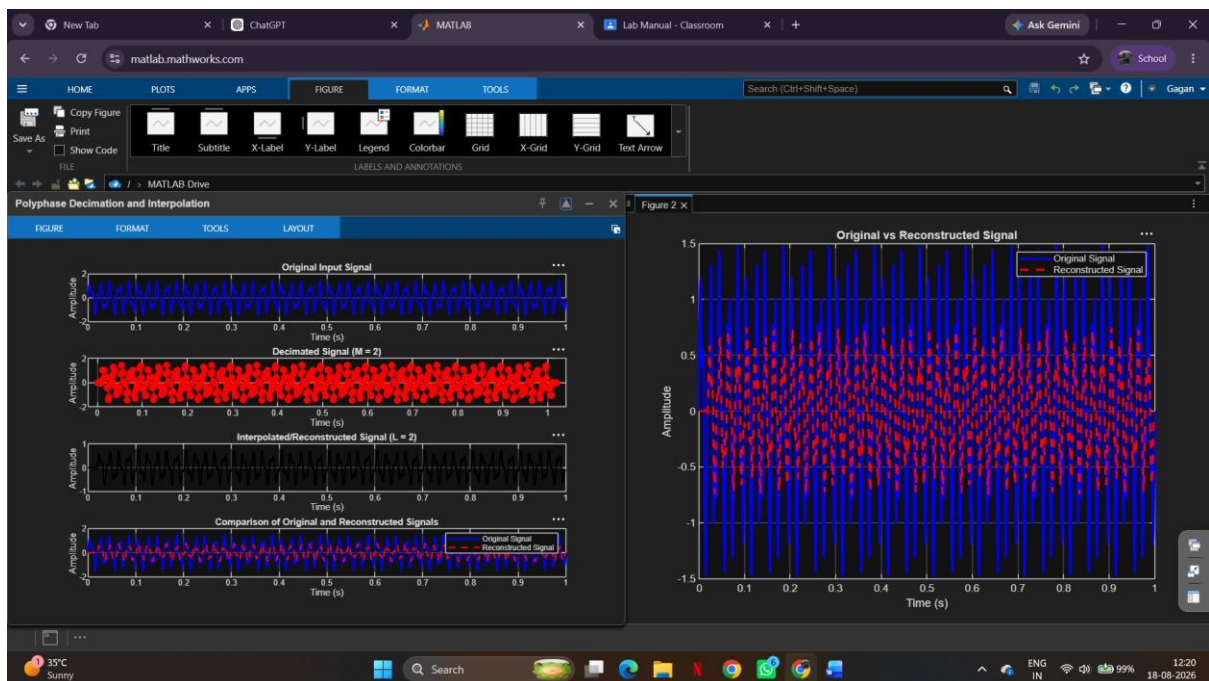
```

```

title('Original vs Reconstructed Signal');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Original Signal', 'Reconstructed Signal');
grid on;

```

Output:



POLYPHASE DECIMATION & INTERPOLATION

Original Signal Length = 1001

Decimated Signal Length = 511

Interpolated Signal Length = 1001

Result :

1. The input signal was successfully decimated by a factor of 2 using a polyphase FIR decimation filter.
2. The input signal was successfully interpolated by a factor of 2 using a polyphase FIR interpolation filter.
3. The original signal was reconstructed from the decimated signal and closely matched the input signal.
4. The polyphase filter structure reduced computational complexity and efficiently performed multirate signal processing operations.